COURSE REGISTRATION SYSTEM USING COMPOSITE DESIGN PATTERN

1. Title Page

Course Name: Object-Oriented Programming (Java)
Project Title: Course Registration System using Composite Design Pattern
Student Name: Amrita Jaiswal
Roll Number: 75
Instructor Name: Soumadip Biswas
Semester: Spring 2026

2. Problem Statement

Educational institutions often deal with complex course structures consisting of multiple modules, sub-modules, and learning units. Traditional systems treat courses as flat entities, which limits flexibility in representing hierarchical relationships among course components. This creates challenges in organizing, managing, and visualizing course content effectively.

The objective of this project is to design and implement a Course Registration System in Java that models courses and their hierarchical structure using the Composite Design Pattern. This approach allows both individual modules and grouped modules to be treated uniformly. The system enables students to register for courses, view structured course content, and manage their registrations efficiently.

The project focuses on applying Object-Oriented Programming (OOP) concepts such as abstraction, inheritance, polymorphism, and encapsulation. It also incorporates exception handling, file handling for data persistence, and modular design principles. The final system aims to be scalable, maintainable, and easy to extend for future enhancements.

3. SYSTEM DESIGN

```
                    CourseComponent (Abstract)
                    -------------------------
                    - name : String
                    + displayDetails()
                    + getCredits()

                              ▲
                              |
            ┌─────────────────┼─────────────────┐
            |                                   |
        Course                            Module / SubModule
    (Composite Class)                 (Leaf Classes)

 Student ------> Course
 Student ------> RegistrationManager
 RegistrationManager ------> Course
 FileHandler ------> Student
```

DESCRIPTION OF MAJOR CLASSES

CourseComponent

This is an abstract class that acts as the base for both composite and leaf nodes. It defines common methods such as displayDetails() and getCredits().

Course

This class represents a complete course and acts as a composite. It can contain multiple modules or sub-modules. It uses a collection (ArrayList) to store child components.

Module and SubModule

These are leaf classes in the composite structure. They represent the smallest units of a course and cannot contain further components.

Student

This class represents a student and maintains a list of registered courses. It provides methods to add and display courses.

RegistrationManager

This class handles the business logic of course registration. It ensures that duplicate registrations are avoided and manages interaction between student and course.

FileHandler

This class is responsible for saving student registration data into a file. It ensures persistence of data.

PACKAGE STRUCTURE

model → Core classes (Course, Module, Student, etc.)

service → Registration logic

util → File handling

exception → Custom exceptions

main → Entry point

INTERACTION FLOW

1. User starts the program
2. Menu options are displayed
3. User selects an action (register/view/save)
4. RegistrationManager processes the request
5. Student object is updated
6. FileHandler saves data if selected

4. OOP CONCEPTS DEMONSTRATED

1. Abstraction

Where used: CourseComponent class
Why used: To define a common interface

```
public abstract class CourseComponent {
    protected String name;

    public CourseComponent(String name) {
        this.name = name;
    }

    public abstract void displayDetails(String indent);
    public abstract int getCredits();
}
```

2. Inheritance

Where used: Module, Course, SubModule

```
public class Module extends CourseComponent {
    private int credits;

    public Module(String name, int credits) {
        super(name);
        this.credits = credits;
    }
}
```

Explanation:
Inheritance allows reuse of code and establishes hierarchy.

3. Polymorphism

Where used: displayDetails() method

```
for (CourseComponent c : components) {
    c.displayDetails(indent + "  ");
}
```

Explanation:
The same method behaves differently depending on object type (Course vs Module).

4. Encapsulation

Where used: Student class

```
private List<Course> courses;


public List<Course> getCourses() {
    return courses;
}
```

Explanation:
Data is protected and accessed through methods.

5. Exception Handling

Where used: RegistrationManager

```
if (student.getCourses().contains(course)) {
    throw new RegistrationException("Already registered!");
}
```

Explanation:
Prevents invalid operations and improves system reliability.

5. IMPLEMENTATION HIGHLIGHTS

Composite Pattern Implementation

```
public void addComponent(CourseComponent c) {
    components.add(c);
}
```

Justification:
Allows dynamic addition of modules and supports hierarchical structure.

Recursive Display

```
c.displayDetails(indent + "  ");
```

Justification:
Provides a clean hierarchical view of courses.

Menu-driven System

```
while (true) {
    System.out.println("1.Register 2.View 3.Save 4.Exit");
}
```

Justification:
Makes the application interactive and user-friendly.

File Handling

```
FileWriter fw = new FileWriter("registration.txt");
```

Justification:Stores data persistently, ensuring information is not lost.

7. TESTING & ERROR HANDLING

Test Cases Considered

- Registering a course successfully
- Viewing registered courses
- Saving data to file

Edge Cases

- Registering the same course multiple times
- Viewing courses when none are registered
- Invalid menu inputs
-

Failure Scenarios

- File writing errors
- Invalid user input
- Duplicate registration attempts
- All handled using try-catch blocks and custom exceptions.

7. GIT WORKFLOW

fix: resolve name access issue in Module display method AMRITAJAISWA committed 12 hours ago	4a2c642
feat: add Module class with credits and display functionality AMRITAJAISWA committed 13 hours ago	f2e7c89
Fix SubModule to use getter for name and ensure proper encapsulation AMRITAJAISWA committed 13 hours ago	9fa906d
Enhance RegistrationException with default and parameterized constructors AMRITAJAISWA committed 13 hours ago	327281c
Fix RegistrationManager syntax errors and improve registration logic AMRITAJAISWA committed 14 hours ago	05d48b6
Fix FileHandler syntax errors and implement file saving functionality AMRITAJAISWA committed 14 hours ago	d816e6e
Added abstract CourseComponent class for composite pattern AMRITAJAISWA committed 15 hours ago	7f815f2

Commits on Mar 28, 2026

Initial commit - added main functionality AMRITAJAISWA committed 15 hours ago	64e7f43
Added pro.java file AMRITAJAISWA committed 17 hours ago	8283c19
add deadline github-classroom[bot] authored 18 hours ago	Verified 751cf5e

Development Progression

1. Created initial project structure
2. Added abstract base class
3. Implemented Module class
4. Implemented Course composite
5. Added SubModule
6. Added Student class
7. Added RegistrationManager
8. Implemented Exception handling
9. Added File handling
10. Added Menu system

This shows a structured and incremental development process.

CONCLUSION & FUTURE SCOPE

Conclusion

The project successfully demonstrates the implementation of the Composite Design Pattern to manage hierarchical course structures. It effectively applies OOP principles and provides a modular and scalable solution.

The system is easy to maintain, extend, and understand.

Future Scope

- Graphical User Interface using Java Swing or JavaFX
- Integration with database (MySQL)
- Web-based application
- Multi-user support system
- Cloud-based storage

Appendix

Project Title

Course Registration System using Composite Design Pattern for Course Modules Problem Statement

Educational institutions manage multiple courses, each containing several modules, sub-modules, and learning units. Traditional course registration systems often treat courses as flat structures, making it difficult to represent hierarchical course content and flexible module organization. This project proposes a Javabased Course Registration System that models courses and modules using the **Composite Design Pattern**, allowing both individual modules and grouped modules to be treated uniformly. The system will enable students to register for courses, view course structures, and access module details in a hierarchical format. The project emphasizes clean object-oriented design principles, modular architecture, and proper exception handling. By applying OOP concepts such as abstraction, inheritance, and polymorphism, the system will provide a scalable and maintainable approach for managing course structures while demonstrating disciplined coding practices and version control usage.

Target User

University students and academic administrators who need to manage or register for structured courses with multiple modules.

Core Features

Student course registration and withdrawal

Display hierarchical course/module structure

Add or remove course modules

View module details and prerequisites

Maintain registered course records using collections File storage for saving student registration data Input validation and exception handling

OOP Concepts to be Used

Abstraction

Abstract class/interface for CourseComponent representing both courses and modules.

Inheritance Course, Module, and SubModule classes inherit from the base CourseComponent.

Polymorphism Common operations like displayDetails() and getCredits() behave differently for courses and modules.

Exception Handling Handles invalid registrations, duplicate registrations, and file I/O errors.

Collections / Threads (if applicable) Java Collections (ArrayList, HashMap) will manage students, courses, and registrations. File handling will persist data.

Proposed Architecture Description

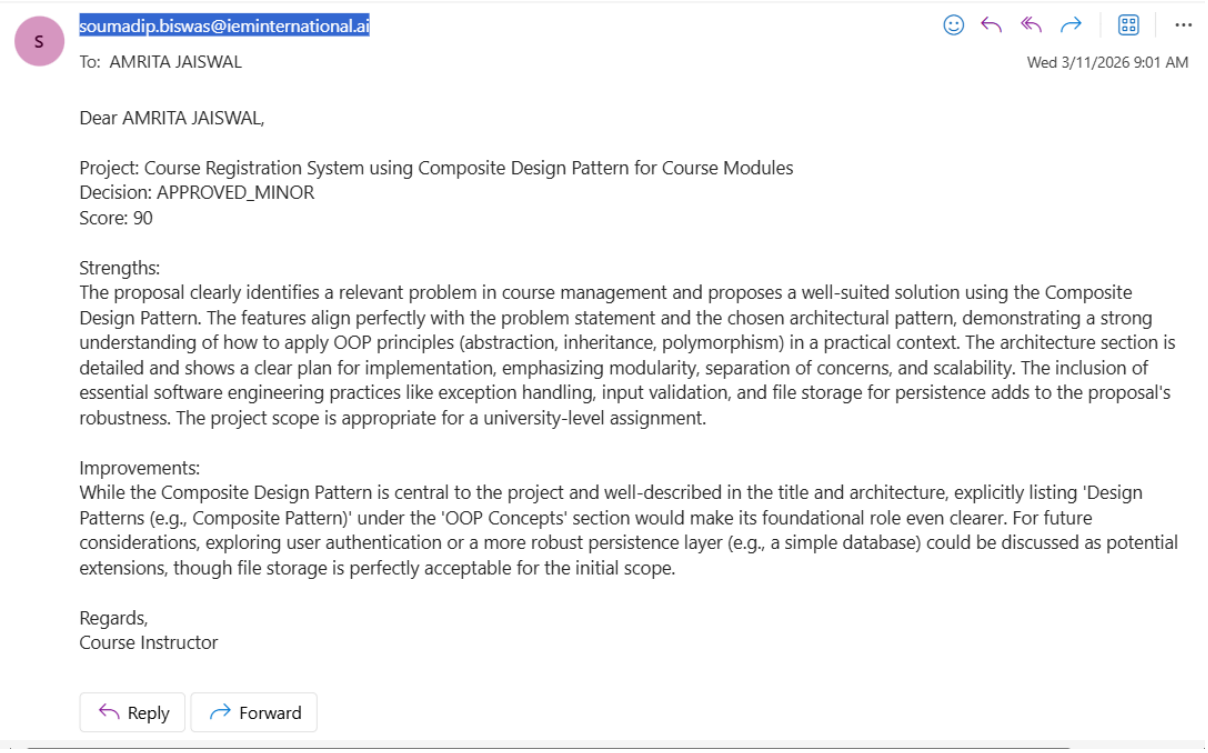
The system will follow a modular object-oriented architecture built around the **Composite Design Pattern**. A base component interface (CourseComponent) will define common operations

for both courses and modules. Composite classes (Courses containing Modules) and leaf classes (individual modules) will allow hierarchical course representation. A Student class will

manage registration actions, while a RegistrationManager will coordinate course enrollment using Java collections. File handling will store registration records. This design ensures

scalability, code reuse, and clear separation of responsibilities while demonstrating key Java OOP principles.

SCREENSHOT



S soumadip.biswas@ieminternational.ai

To: AMRITA JAISWAL

Wed 3/11/2026 9:01 AM

Dear AMRITA JAISWAL,

Project: Course Registration System using Composite Design Pattern for Course Modules
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly identifies a relevant problem in course management and proposes a well-suited solution using the Composite Design Pattern. The features align perfectly with the problem statement and the chosen architectural pattern, demonstrating a strong understanding of how to apply OOP principles (abstraction, inheritance, polymorphism) in a practical context. The architecture section is detailed and shows a clear plan for implementation, emphasizing modularity, separation of concerns, and scalability. The inclusion of essential software engineering practices like exception handling, input validation, and file storage for persistence adds to the proposal's robustness. The project scope is appropriate for a university-level assignment.

Improvements:
While the Composite Design Pattern is central to the project and well-described in the title and architecture, explicitly listing 'Design Patterns (e.g., Composite Pattern)' under the 'OOP Concepts' section would make its foundational role even clearer. For future considerations, exploring user authentication or a more robust persistence layer (e.g., a simple database) could be discussed as potential extensions, though file storage is perfectly acceptable for the initial scope.

Regards,
Course Instructor

Reply Forward